

960121

Security & Auth Logic

Session 06



Objective

- For **Session 6: Security & Auth Logic (User Identity)**, we are tackling the most sensitive part of any application. This is where "Critical Thinking" isn't just a soft skill—it's the difference between a successful business and a legal disaster.
- In this session, we will learn that **Identity** is a "**Token of Trust**" that the server issues after a rigorous check.

Architecture of Trust

The **Architecture of Trust** is what transforms a simple "website" into a professional "platform." In an undergrad setting, students often think security is just a "Login Page."

You need to know that security is actually a **multi-layered filter system** that validates every single data packet.

Authentication vs. Authorization

- **AuthN**: Are you who you say you are? (Login).
- **AuthZ**: Do you have permission to do this? (e.g., Only an Admin can change prices).
- Authentication (e.g., logging in) must always occur before authorization (e.g., accessing files).
- Authentication uses credentials like passwords, OTP or biometrics.
- Authorization uses policies like role-based access control (RBAC).

1. The Three Layers of Defense

- **Layer 1: Encryption in Transit:** Before data reaches the server, it must be encrypted via HTTPS/TLS. If a student's login sends a password over HTTP (not HTTPS), the "Gatekeeper" is useless because the password was stolen on the way to the door.
- **Layer 2: Authentication (The ID Check):** Identifying who is making the request. We don't store passwords; we store Hashes (irreversible fingerprints).
- **Layer 3: Authorization (The Keycard):** Just because I am "John" doesn't mean I can edit "Sarah's" profile. The server must check the Token permissions for every single request.

2. Hashing: The "One-Way" Logic

Encryption (reversible) vs. **Hashing** (irreversible)

- **The Business Logic:** If our database is hacked, we don't want the hackers to see "Password123."
- **The Process:**
 - 1. User signs up: **Password** → **Hash Function** → Stored **xyz789**.
 - 2. User logs in: **Input** → **Hash Function** → Compare with **xyz789**.
- **Critical Thinking:** We never "decrypt" a password. We only compare fingerprints.

Encryption

- **Encryption** is the process of **converting readable data** (plaintext) **into an unreadable scrambled format** (ciphertext) using **cryptographic algorithms** and **keys**.
- Algorithms scramble data, which requires a specific digital key—a set of mathematical values—to unlock or decrypt it back into a readable format.
- It ensures data confidentiality, allowing only authorized parties with the correct key to decrypt and access the information, protecting it from theft, tampering, and unauthorized access.

Encryption

Types of Encryption:

- **Symmetric Encryption**: Uses the same key for both encryption and decryption, offering speed.
- **Asymmetric Encryption**: Uses a pair of keys—a **public key** for encryption and a **private key** for decryption—which is more secure.

Hashing

- **Hashing** is the process of transforming any size of data (input) into a fixed-length string of characters (output) using a **mathematical formula** known as a hash function.
- It is a one-way, non-reversible process used for data indexing, fast data retrieval, and ensuring data integrity. Common uses include storing passwords securely and verifying file integrity.

Hashing

- **One-Way Function:** Hashing cannot be reversed to find the original data, ensuring high security.
- **Fixed-Length Output:** No matter the size of the input (a single letter or a large file), the output hash has a set, consistent length.
- **Deterministic:** The same input always produces the exact same hash.
- **Avalanche Effect:** A tiny modification in the input (like changing a single character) results in a completely different hash output.

3. JWT (JSON Web Tokens): The Passport System

In a "Stateless" backend (Session 5), the server doesn't remember you. The JWT is the solution.

- JSON Web Tokens (JWT) are an open, industry-standard (RFC 7519) method for securely transmitting information between parties as a compact, URL-safe JSON object.
- They are primarily used for authentication and authorization, allowing servers to verify token integrity using a secret or public/private key pair, eliminating the need for session state.

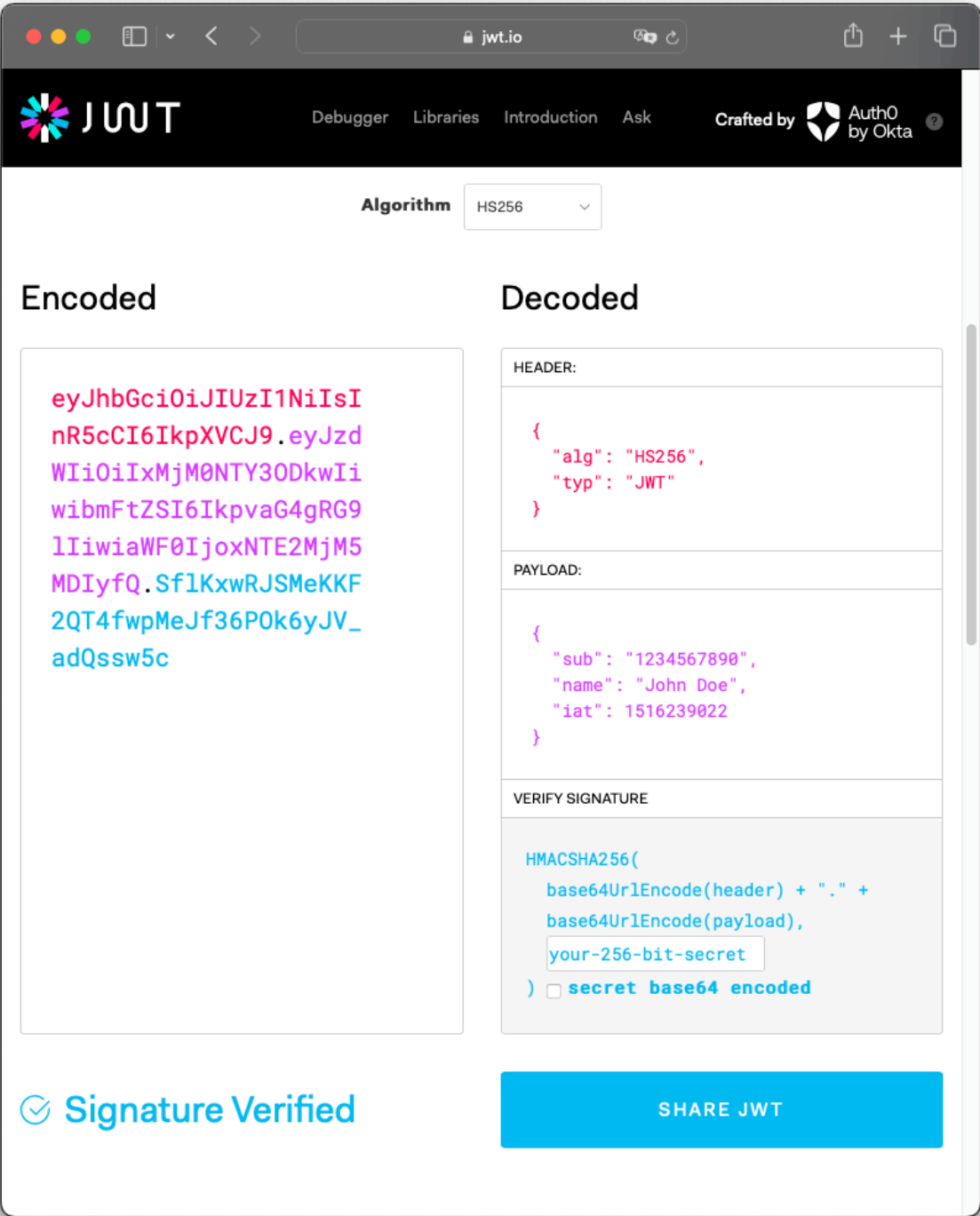
JWT

Structure: Consists of three parts separated by dots (.):

- **Header:** Indicates which algorithm was used.
- **Payload:** Contains non-sensitive user data (e.g., `user_id: 101, role: 'customer'`).
- **Signature:** This is the most important part. It is a mathematical proof that the server created this token. If you try to change your `user_id` in the browser, the Signature will no longer match, and the Gatekeeper will reject the request.

JWT

- **Self-Contained**: The token contains all necessary user information, reducing the need to query the database repeatedly.
- **Compact**: Small size allows them to be sent via HTTP headers, URLs, or POST parameters.
- **Usage**: Often used in single-page applications (SPAs), mobile apps, and APIs where tokens are sent in the Authorization header using the Bearer schema.



JWT

- JWT typically looks like the following:

xxxxxx.yyyyyy.zzzzzz

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaXNTb2NpYWwiOnRydWV9.
4pcPyMD09o1PSyXnrXCjTwXyr4Bsezdi1AVTmud2fU4



Referance

- **Introduction to JSON Web Tokens**
(<https://www.jwt.io/introduction#what-is-json-web-token>)

Class Activity:

Mapping the Login Handshake

1. The Scenario:

- **The Challenge:** A user wants to log in. We have a "Users" list on the server.
- **Goal:** We need to verify the password, create a session, and make sure the "Add to Cart" function knows *which* user is shopping.

The "Login Handshake"

Step	Entity	Action	Logic/Security Note
1	Client	Sends Credentials (POST)	Must be sent over HTTPS.
2	Server	Validates & Hashes	Checks DB for the user.
3	Server	Signs JWT	Creates a "Signed Passport."
4	Client	Stores Token	Usually in <code>localStorage</code> or <code>Cookies</code> .
5	Client	Attaches Token to Headers	Every future request includes <code>Authorization: Bearer <token></code> .

Mapping the Login Handshake

2. (15 min.) UML:

- Draw a **Sequence Diagram** for the login process.
 - Do not forget the **API Gateway**
- **Logic Check:** You must draw the "Back-and-Forth":
 - Client sends `email/password`.
 - Server checks database.
 - Server creates a `Token`.
 - Server sends `Token` back to Client.
 - Client stores `Token` (Continuity logic from Session 4!).

Mapping the Login Handshake

3. (5 min.) Class Evaluation:

1. **Frontend → Backend**: `POST /login {credentials}`
 - **Critical Thinking**: Label this arrow "Encrypted via HTTPS."
2. **Backend → Database**: `findUser(email)`
 - **Logic**: The server doesn't have the data; it must ask the DB for the stored hash.
3. **Backend (Internal)**: `bcrypt.compare()`
 - **Security**: This is the "Gatekeeper" moment. We compare the user's input with the hash.

Mapping the Login Handshake

3. (5 min.) **Class Evaluation:**

4. **Backend** → **Backend**: `jwt.sign(payload, secret)`

- **Architecture:** The server creates a "Passport" (Token).

5. **Backend** → **Frontend**: `Response{token: 'xyz...'}`

- **Continuity:** The frontend receives the "Key."

6. **Frontend** (Internal): `localStorage.setItem('token')`

- **Persistence:** The user remains logged in even after refreshing (reconnecting to Session 4).

Mapping the Login Handshake

3. (5 min.) **Class Evaluation:**

■ **Critical Thinking Question:**

- If an attacker intercepts the password in step 1, what happens?
- How does HTTPS protect this?
- Where is the password stored?
- Who holds the "Secret Key" used to sign the Token?
- What happens if the `findUser` step returns null?

Architecting the API

4. (5 min.) GenAI Prompt Engineering:

Prompt: (prepare auth_user.json)

“Generate a JSON file of 10 registered users with username (email), password (hash with MD5), first name, and date of registration. Also, give me the lists of pare email and plaintext password for login testing.”

Architecting the API

4. (10 min.) GenAI Prompt Engineering:

Prompt:

“Act as a Security Engineer. Write a Node.js/Express POST route for `/api/login`. It should receive an email and a password. Check if the email is not in my database, return 401 Unauthorized status. If it exists, use a library like `bcrypt` to compare the submitted password with a hashed one in my database. If it matches, use `jsonwebtoken` to sign a token containing the user's ID. Return the token with a 200 status.”

Architecting the API

5. (20 min.) **Integration:**

- Open your project in VS Code.
- Add `jsonwebtoken` and `bcrypt` to your project.
- Integrate the login route into your backend server.
- **Do not forget** to initialize your backend using `npm init`.

Architecting the API

6. (20 min.) Debugging & Testing:

- **The "Invalid User" Test:** Try logging in with the wrong password. Does the server leak info?
- **Critical Thinking:**
 - Why should the error say 'Invalid email or password' instead of 'Email not found'?

Architecting the API

7. (5 min.) Version Control:

- Terminal Command:

- `git add`

- `git commit -m "feat: implement JWT-based authentication and password hashing."`

- `git push.`

- VS Code:

- GitHub Desktop:

Key Takeaway

- **Takeaway 1:** Identity is just another piece of **State**.
- **Takeaway 2:** The Token is the "Key" to the Gatekeeper. Without it, the server closes the door.
- **Takeaway 3: Security is a Process, not a Feature.** We don't "add security" at the end; we build it into the Request/Response map from the start.

Tonight Work (5 points)

Implement your register feature, follow this **Logic**:

- Request the name of the user, email for username, and password with at least 8 characters, with at least one special character (like !, @, #, \$, %, ^, &, *) and one uppercase character.
- Check the password conditions on the frontend.
- Check for existing username on the backend using `auth_user.json`.
- Store new register uses in `auth_user.json`.

Tonight Work (5 points)

1. Write the "Contract" Table.
2. Write an **Activity Diagram** and a **Sequence Diagram** for user registration.
3. Write GenAI Prompt using the "Contract" table + Diagrams.
 - `Express.js` route for this request.
 - `register.js` for this logic.

Ensure your code has comments explaining how it works!!

Tonight Work (5 points)

Submission:

1. Digital version (.pdf or image) of your:
 - Contract table
 - Diagrams
 - GenAI Prompt(s)

Placed them in `Documentation/register`

2. `git commit -m "feat: tonight work"`
3. GitHub commit before Tuesday 11.59 PM